DexCapTwist: Synthetic Asset Generation with Hybrid Primitive Control for Dexterous Screw-Cap Manipulation

***Abstract*—Dexterous screw-cap manipulation requires a robot to accommodate substantial container-geometry variation while repeatedly alternating between contact-rich twisting and finger repositioning. Existing articulated-object resources rarely provide task-specific bottle assets at scale, while monolithic policies do not explicitly model this cyclic structure. We present DexCapTwist, a unified framework comprising the DexTwistSet dataset and the CycleTwist controller. DexTwistSet contains 4,015 validated, simulation-ready articulated bottle assets. Its construction pipeline extracts paired body–cap contours from internet images, models and expands their shape distribution with ContourVAE, and generates URDF/USD assets with bimanual grasp initialization. CycleTwist decomposes unscrewing into a learned twisting primitive and a deterministic reset primitive. A DAgger-refined Switching Discriminator identifies when to reset the hand and when to resume twisting, enabling stable long-horizon execution. Simulation experiments show that CycleTwist improves average cap rotation and twisting velocity over competitive baselines across original, interpolated, and randomly sampled geometries. On 20 everyday containers, the policy achieves a 78.5% zero-shot success rate without real-world fine-tuning. Code, assets, and videos will be released at `[GitHub link to be added]`.**

***Index Terms*—Articulated-object dataset, dexterous manipulation, reinforcement learning, sim-to-real transfer, screw-cap unscrewing**

***Note to Practitioners*—Screw-cap opening appears in packaging, laboratory automation, household assistance, and other settings where container geometry changes frequently. Developing such systems in simulation is attractive, but manually modeling the body, cap, joint properties, and feasible two-hand initialization for every container is expensive. DexTwistSet provides practitioners with 4,015 validated articulated bottle assets that can be used directly for reinforcement-learning pretraining, geometry-based stress testing, and benchmark evaluation. The associated construction workflow can also be applied to internet images when assets for a new container family are required. For execution, CycleTwist treats unscrewing as repeated twist–reset cycles: a learned policy generates contact-rich rotation, a deterministic primitive restores a favorable hand configuration, and a learned discriminator switches between them. This modular design makes the recovery behavior explicit and easier to inspect than a single end-to-end policy. In our hardware evaluation, a policy trained entirely in simulation transfers without real-world fine-tuning and succeeds on 78.5% of attempts across 20 everyday containers. Deployment still requires compatible two-hand grasping, adequate cap friction, and collision-safe reset trajectories.**

I. Introduction

CONTAINERS with screw caps appear widely in industrial, household, and service-robot scenarios. Opening them with dexterous robotic hands is a representative contact-rich manipulation problem. One hand must stabilize the container body. The other hand must apply tangential force to the cap, maintain contact under frictional uncertainty, and re-position its fingers after each partial turn. These requirements make screw-cap manipulation more cyclic and contact-sensitive than rigid-object grasping.

Large-scale datasets have substantially improved dexterous grasping and general manipulation. Representative examples include DexGraspNet [1], [2], Dex1B [3], and DexVLG [4]. Articulated-object resources [5], [6] and dexterous manipulation benchmarks [7], [8] cover many doors, drawers, faucets, and tool-like objects. However, screw-cap manipulation requires a different asset profile: articulated cap-body geometry, cap-joint resistance, and validated bimanual grasp initialization must be available at scale. This gap limits reinforcement-learning training across the large geometry variation found in everyday containers.

Control also remains challenging. Human cap unscrewing naturally alternates between two phases. During twisting, the fingers maintain contact and generate tangential torque. During reset, the hand returns to a favorable pre-grasp configuration for the next turn. Prior lid- or cap-twisting methods [9], [10] have demonstrated promising results, but monolithic end-to-end policies must represent contact-rich torque generation and non-contact hand recovery within a single action distribution. This coupling can reduce stability and sample efficiency, especially under broad object-geometry variation. The long-horizon cyclic structure of unscrewing therefore calls for phase-aware control.

This paper presents **DexCapTwist**, a unified framework for dexterous screw-cap manipulation. First, **DexTwistSet** is a task-specific dataset of simulation-ready articulated bottle assets. Its construction pipeline starts from internet images: paired body–cap contours are extracted, modeled with **ContourVAE**, expanded through latent-space interpolation and random sampling, then converted into articulated URDF/USD assets for Isaac Sim. Each validated asset includes randomized body geometry, cap geometry, visual appearance, cap-body joint resistance, and bimanual initial grasps. Second, **CycleTwist** learns cap unscrewing with hybrid primitive control. A geometry-aware reinforcement-learning policy handles the twisting phase. A deterministic reset primitive recovers the hand configuration. A DAgger-refined **Switching Discriminator** triggers phase transitions during cyclic execution. Third, the complete system is evaluated in simulation and on a physical bimanual platform to test geometric generalization and zero-shot transfer.

The main contributions of this paper are three-fold:

1) **DexTwistSet**, a dataset of 4,015 validated articulated URDF/USD bottle assets for dexterous screw-cap manipulation and reinforcement learning. Its internet-image-driven construction pipeline extracts body–cap contours and trains ContourVAE for scalable contour synthesis.

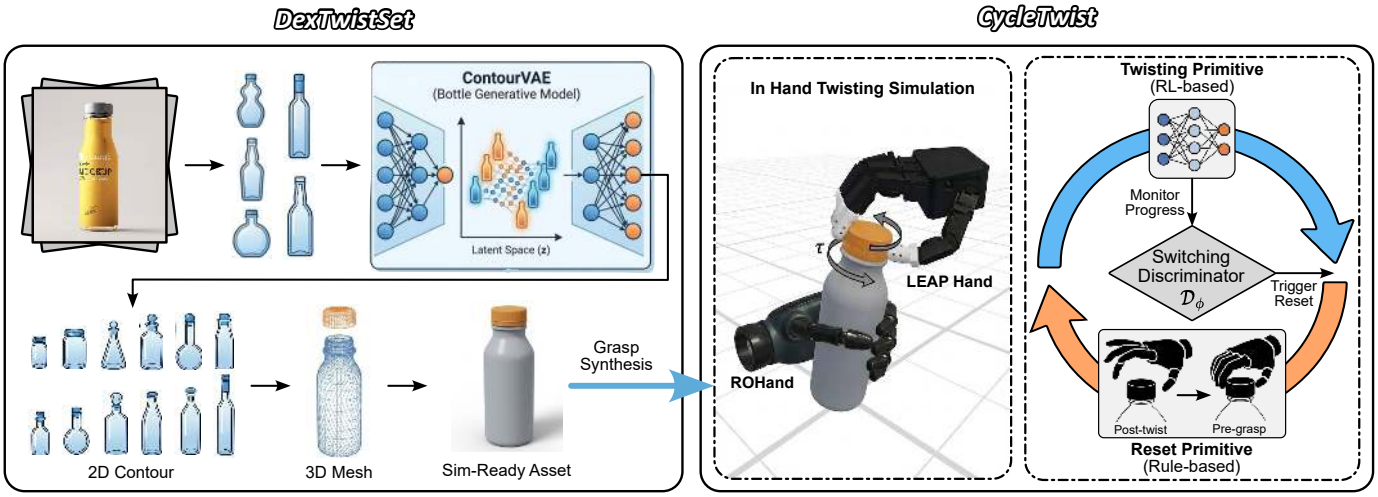

Fig. 1. System overview of DexCapTwist. The construction pipeline for **DexTwistSet** (left) converts 2D images into diverse, simulation-ready 3D bottle assets through the ContourVAE generative model. **CycleTwist** (right) uses a hybrid control architecture with a Switching Discriminator D_ϕ to coordinate a reinforcement-learning twisting primitive and a rule-based reset primitive for stable, long-horizon bimanual manipulation.

2) **CycleTwist**, a hybrid primitive-control framework that combines a geometry-aware learned twisting primitive with a deterministic reset primitive. A DAgger-refined switching discriminator coordinates cyclic transitions between twisting and reset.

3) Systematic simulation and real-world validation showing that scalable synthetic asset diversity and hybrid primitive control improve average cap rotation, twisting velocity, and zero-shot physical deployment on everyday containers.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the construction of the DexTwistSet dataset. Section IV details the CycleTwist control framework. Section V reports simulation and real-world experiments. Section VI concludes the paper.

II. Related Work

This section reviews the two bodies of work most directly related to DexCapTwist. The first concerns whether existing dexterous manipulation datasets, benchmarks, and articulated-object assets can support screw-cap policy learning. The second concerns robotic twist manipulation, including cap screwing, bottle-cap unscrewing, key turning, and bolt screwing. This organization follows the two technical questions of this paper: what assets are missing for training across container geometry, and what control structure is needed to execute repeated twist-reset cycles.

A. Dexterous Manipulation Datasets and Articulated Assets

Robotic manipulation datasets have grown rapidly, but most were not designed for cap-twisting interaction. DexYCB [11], ContactDB [12], and ContactPose [13] provide valuable data for rigid-body hand-object interaction. DexGraspNet [1], [2], Dex1B [3], and DexVLG [4] further increase scale for dexterous grasping, demonstrations, or vision-language grasping. Recent TASE work on fine-grained grasp detection also shows the value of part-level affordance datasets for task-aware grasping [14]. These resources support grasp acquisition, affordance reasoning, and imitation learning, but they do not provide interactive articulated cap-body assets with task-specific joint resistance.

Articulated-object resources and robot-learning benchmarks cover a broader range of object mechanisms. PartNet-Mobility [5], Infinigen-Sim [6], Meta-World [15], ManiSkill [16], and ManiSkill2 [17] include objects such as doors, drawers, faucets, and cabinets. DexArt [7] and DexMachina [8] extend articulated-object manipulation toward dexterous hands and bimanual settings. However, screw-cap manipulation requires a narrower asset specification: paired body–cap geometry, cap-body joint resistance, and validated bimanual grasp initialization must be available for many bottle shapes. Existing datasets and benchmarks rarely combine these requirements, leaving a task-specific asset gap for large-scale RL training in dexterous cap manipulation.

B. Learning-Based Twist Manipulation

Twisting has long been studied as a contact-rich manipulation primitive. Journal work on dexterous in-hand manipulation has combined analytical models with learning to derive manipulation primitives for adaptive hands [18], and TASE work has integrated learning from demonstrations with reinforcement learning for nonprehensile manipulation [19]. Early tactile cap-screwing work used tri-axial tactile data to adapt robotic finger trajectories during bottle-cap screwing [20]. Kernel dynamic policy programming was later applied to a PAM-driven humanoid hand for unscrewing a bottle cap with tactile sensors [21]. Demonstration learning has also been used for robot screwing skills across bolts, plastic bottle caps, and faucets [22]. These studies establish the importance of contact sensing, torque transmission, and trajectory adaptation, but they typically address constrained objects, non-dexterous end effectors, or single-task execution rather than large-scale dexterous generalization.

Recent work has moved closer to dexterous cap and twist manipulation. Lin et al. [9] trained two robot hands to twist bottle lids with deep RL and sim-to-real transfer. Tac2Motion [10] uses tactile feedback for contact-aware RL

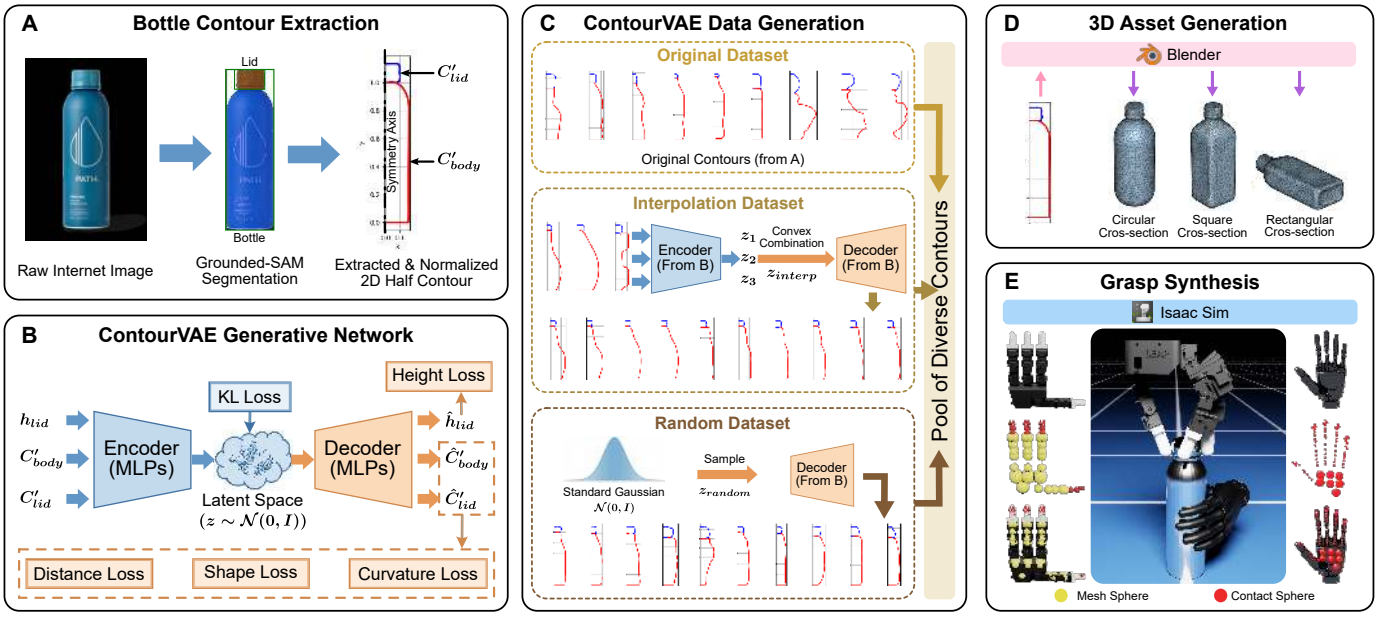

Fig. 2. Construction pipeline for the DexTwistSet dataset. (A) Pre-processing extracts 2D bottle contours from internet images. (B) ContourVAE uses an MLP-based encoder-decoder structure optimized with multi-objective losses. (C) Latent-space interpolation and random sampling expand the contour pool. (D) Blender converts contours into 3D bottle assets. (E) Isaac Sim deployment validates assets and bimanual grasps for large-scale reinforcement learning.

in lid removal. NeuralTouch [23] combines neural descriptors with tactile feedback for precise sim-to-real control, including bottle-lid opening. DexAnyTwist [24] models general dexterous twisting as hybrid manipulation system identification and uses specialized expert policies with learned gating to handle heterogeneous twisting dynamics. DexTwist [25] studies functional retargeting for contact-rich rotational tasks such as cap opening, key turning, and bolt screwing. These methods show that learning-based or teleoperated twist manipulation is feasible. However, most focus on tactile servoing, teleoperation, or monolithic end-to-end policies. They do not jointly address large-scale task-specific screw-cap asset generation and explicit phase-aware twist-reset control for long-horizon unscrewing under broad bottle geometry variation.

III. DexTwistSet Dataset Construction

DexTwistSet is a dataset of validated articulated bottle assets for reinforcement learning. As illustrated in Fig. 2, its construction pipeline converts unstructured internet images into simulation-ready assets in three stages. It first extracts paired body–cap contours from internet bottle images. It then learns and expands their latent distribution with ContourVAE. Finally, it converts the generated contours into articulated 3D assets, deploys them in Isaac Sim, and validates bimanual grasp initialization. Each validated asset includes meshes, grasp poses, and URDF/USD files for systematic evaluation across diverse bottle geometries.

A. Bottle Contour Generation

Generating diverse articulated bottle assets is difficult because each model must include plausible body geometry, cap geometry, and a cap-body joint. Internet images provide broad shape coverage, but they are unstructured and cannot be used directly in simulation. We therefore convert internet bottle images into paired contour data, train ContourVAE to learn the contour distribution, and use the decoder to generate physically plausible bottle contour samples.

1) Bottle Contour Extraction: First, we gather bottle images from the internet using 18 curated keywords that span common container types, such as “beverage bottle”, “vacuum flask”, and “water bottle”. This collection covers a wide range of silhouettes and cap styles.

For each image, Grounded-SAM [26] provides automatic instance segmentation by combining Grounding DINO with Segment Anything. The prompts “bottle” and “lid” isolate the body and cap components. The “bottle” mask represents the union of body and cap regions. The body mask is obtained by subtracting the “lid” mask from this union, while the cap mask is given by the “lid” output. We then apply OpenCV’s Suzuki85 algorithm [27] to extract 2D contours from these masks, obtaining the body contour C_{body} and cap contour C_{lid}.

Internet bottle images often contain occlusions, cluttered backgrounds, or incomplete views. Grounded-SAM can therefore produce imperfect body or cap masks. We apply geometric post-processing to filter invalid contours and improve contour quality.

Let B_{bottle} and B_{lid} denote the bounding boxes of the “bottle” and “lid” masks. A contour pair is retained when four geometric checks are satisfied: (a) B_{lid} is contained within B_{bottle}; (b) the maximum height difference between the two boxes is below a specified tolerance; (c) the distance between their vertical symmetry axes is below a specified tolerance; and (d) the lid-to-bottle height ratio is below a specified tolerance.

After filtering, we extract the right-side half contours of C_{body} and C_{lid} for revolution modeling. The half contours are centered at the bottom centers of the body and cap, respectively. We then apply moving average smoothing, use B-spline interpolation to resample each contour to a fixed number N of 2D points, and normalize all coordinates by the body

height.

2) ContourVAE Generative Network: Given the extracted body–cap contour pairs, ContourVAE learns a latent distribution over bottle geometries. The model follows a variational autoencoder design and generates novel body–cap contour pairs from latent samples.

The inputs are the processed body contour ($C'_{\text{body}} \in \mathbb{R}^{N\times 2}$), cap contour ($C'_{\text{lid}} \in \mathbb{R}^{N\times 2}$), and cap height ($h_{\text{lid}} \in \mathbb{R}$). Dedicated MLP layers extract features from each input. A shared MLP fuses these features and maps them to a latent vector z. The decoder uses MLP layers to reconstruct the body contour, cap contour, and cap height.

ContourVAE is trained with KL divergence, height, distance, shape, and curvature smoothness losses. Quantities with hats $\hat{(\cdot)}$ denote predictions; quantities without hats denote ground truth. The KL divergence regularizes the latent space toward a standard Gaussian distribution:

$$L_{KL} = -0.5 \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2). \tag{1}$$

The height loss constrains the cap height:

$$L_h = \text{MSE}(h_{lid}, \hat{h}_{lid}). \tag{2}$$

The distance loss minimizes the distances between predicted and ground-truth contour points for both body and cap:

$$L_d = \frac{1}{N}\left(\sum \|\hat{p}_i^{\text{body}} - p_i^{\text{body}}\|^2 + \sum \|\hat{p}_i^{\text{lid}} - p_i^{\text{lid}}\|^2\right), \tag{3}$$

where p_i^{body} and p_i^{lid} are ground-truth contour points, and $\hat{p}_i^{\text{body}}$ and $\hat{p}_i^{\text{lid}}$ are reconstructed points.

Shape loss employs bidirectional Hausdorff distance to ensure geometric fidelity of body and cap contours with respect to their ground truths:

$$L_s = \sum_{k\in\{\text{body,lid}\}} \max(d(\hat{C}'_k, C'_k), d(C'_k, \hat{C}'_k)), \tag{4}$$

where $d(A, B) = \max_{a\in A} \min_{b\in B} \|a - b\|$ is the directed Hausdorff distance.

The curvature smoothness term penalizes high-frequency oscillations in the reconstructed body and cap contours:

$$L_c = \sum_{k\in\{\text{body,lid}\}} \frac{1}{N-2} \sum_{i=1}^{N-2} \|\hat{p}^k_{i+2} - 2\hat{p}^k_{i+1} + \hat{p}^k_i\|^2. \tag{5}$$

The overall loss function is thus formulated as:

$$L = w_{KL}L_{KL} + w_h L_h + w_d L_d + w_s L_s + w_c L_c. \tag{6}$$

We train ContourVAE for 100 epochs on 5,250 contours using Adam with learning rate 1×10^{-4}. The loss weights are $w_{KL} = 1.0$ with annealing, $w_d = 1000$, $w_s = 20$, and $w_c = 10$.

3) ContourVAE Data Generation: After training ContourVAE, we sample the latent space and decode the samples into novel bottle geometries. Three strategies are used to construct the contour pool:

1) Original contours. This subset contains bottle contours extracted from internet-collected bottle images and preserves high fidelity to real container shapes.

2) Interpolated contours. To expand the contour pool while maintaining realism, we use latent-space interpolation to produce new bottle geometries. Three bottle contours are randomly sampled from the original subset, encoded into latent vectors z_1, z_2, and z_3, then combined by a convex combination:

$$z_{\text{interp}} = \sum_{i=1}^{3} w_i z_i, \quad w_i \sim \text{Uniform}(0,1), \quad \sum_{i=1}^{3} w_i = 1. \tag{7}$$

The decoder then maps z_{interp} to a new bottle shape.

3) Randomly sampled contours. To explore the learned latent space, we sample z_{random} from the standard Gaussian distribution and decode it to generate additional bottle shapes.

B. 3D Asset Generation and Grasp Synthesis

1) Asset Generation: The synthesized contour pool is converted into simulation-ready articulated bottles through a 2D-to-3D asset generation pipeline.

First, Blender revolves or extrudes bottle contours to generate body and cap meshes. Because real bottles often have circular, square, or rectangular cross-sections, we probabilistically assign cross-section shapes according to the height-to-radius ratio ρ:

For $\rho > 6$, the shape is always circular.

For $3 \le \rho \le 6$, shapes are selected with probabilities $[p_0, p_1, p_2] = [0.6, 0.2, 0.2]$, yielding circular ($p_0$), square ($p_1$), or rectangular ($p_2$) cross-sections.

For $0 < \rho < 3$, the probability of circular selection adaptively increases:

$$p_0 = 0.6 \cdot \left(\frac{\rho}{3}\right)^{3/2}, \quad p_1 = p_2 = \frac{1}{2} \cdot (1 - p_0). \tag{8}$$

Rectangular shapes are parameterized by a width-to-height aspect ratio sampled uniformly from a bounded interval around unity. This setting avoids extremely thin or elongated rectangles while preserving moderate cross-section diversity observed in real-world bottles.

Bottle dimensions are sampled with scales in $[0.15, 0.25]$ and wall thicknesses in $[0.02, 0.05]$ m. The generated body and cap meshes are assembled in URDF format with a revolute joint and a prismatic joint, which approximates the screw motion while remaining efficient for large-scale training. The models are then imported into Isaac Sim, converted to USD format, and assigned randomized physical and visual properties, including color, roughness, and restitution. To model cap-body resistance, we randomize the friction scale of the cap-body revolute joint.

2) Bimanual Grasp Synthesis: Stable initial grasping underpins bottle operations. We build on BODex [28] to derive bimanual initialization poses for the generated assets and integrate the resulting grasps into DexTwistSet. The right hand is a 16-DOF LEAP Hand responsible for cap rotation. The left hand is a compact ROHand (ROH-AP001) responsible for body stabilization.

For bottles, the left ROHand envelops the body, while the right LEAP Hand fingertip-grasps the cap. BODex optimizes contact by minimizing distances between hand marker points

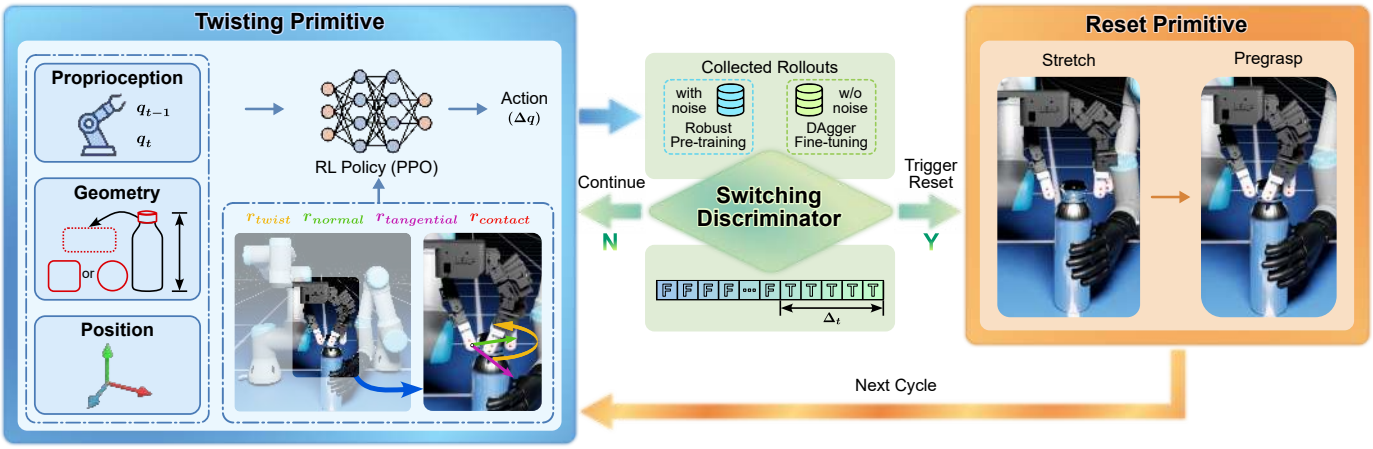

Fig. 3. Architecture of the CycleTwist hybrid control framework. (Left) The **Twisting Primitive** uses PPO to map multimodal observations, including proprioception, object geometry, and spatial position, into joint actions guided by a multi-component reward. (Middle) The **Switching Discriminator** uses collected rollouts for pre-training and DAgger-based fine-tuning, then triggers transitions between primitives. (Right) The **Reset Primitive** executes a deterministic Stretch–Pregrasp sequence to restore the hand-object configuration for subsequent cycles.

and object surfaces. We place contact markers on the inner sides of the left-hand fingers and palm for body grasping, and on the right LEAP Hand fingertips for cap grasping.

After obtaining bimanual grasp poses and joint angles, we validate the generated grasps in the Isaac Lab simulation environment. Validation includes checking whether robot end-effectors can reach the bimanual grasp poses and whether stable grasping can be achieved by both hands.

The validated assets are then used for reinforcement-learning training and evaluation.

IV. CycleTwist Algorithm

CycleTwist is trained with DexTwistSet to solve long-horizon bottle-cap unscrewing, as illustrated in Fig. 3. The task is modeled as a cyclic process with two phases: a twisting phase with finger-contact interaction, and a reset phase that returns the fingers to a favorable configuration. CycleTwist therefore uses hybrid primitive control. A **Twisting Primitive** handles contact-rich manipulation through reinforcement learning. A **Reset Primitive** performs deterministic hand recovery through a scripted trajectory. A DAgger-refined **Switching Discriminator** coordinates autonomous transitions between the two primitives.

A. Twisting Primitive

The core manipulation phase is governed by the Twisting Primitive and modeled as a partially observable Markov decision process.

1) Observation Space: The observation space includes proprioception, bottle geometry, and bottle position. Proprioception contains the right-hand joint positions ($\boldsymbol{q}_t$) and previous target joint positions ($\boldsymbol{q}_{t-1}$). Bottle geometry contains body height, cap shape (encoded as 0 for circular, 1 for square, and intermediate values for rectangular aspect ratios), and sampled cap contour points. Bottle position is represented by the cap position relative to the robot base.

2) Reward Function: To facilitate exploration in this contact-rich setting, we use a fine-grained reward function with the following terms:

- **Twisting Reward**: $r_{\text{twist}} = \theta_t - \theta_{t-1}$, quantifying the incremental counterclockwise rotation of the cap.
- **Tangential Force Reward**: Let $\boldsymbol{f}$ be the fingertip contact force and $\boldsymbol{n}$ be the local cap surface normal. We compute the tangential component $\boldsymbol{f}_\tau = \boldsymbol{f} - (\boldsymbol{f}^\top \boldsymbol{n})\boldsymbol{n}$ and use $r_{\text{tangential}} = \text{clip}(\|\boldsymbol{f}_\tau\|/(\|\boldsymbol{f}\|+\epsilon), 0, 1)$. This bounded term encourages force components that generate effective cap torque.
- **Contact Reward**: For each fingertip point p_f^j, let $d_j = \min_{p_l \in P_l} \|p_f^j - p_l\|_2$ be its closest distance to the cap point set. We define $r_{\text{contact}} = \frac{1}{M}\sum_{j=1}^{M} \exp(-d_j^2/\sigma^2) \in [0, 1]$, promoting dense fingertip–cap proximity without allowing the contact term to become unbounded near zero distance.
- **Regularization Terms**: Penalties on action magnitude and joint acceleration to encourage smooth and energy-efficient motion.

Policy training uses PPO in Isaac Lab with domain randomization for real-world transfer.

B. Reset Primitive

After completing a twisting phase, the system must restore the hand to a repeatable configuration for the next cycle. This requires a reliable trigger from the learning-based twisting primitive to the rule-based reset primitive. We use a **Switching Discriminator** to detect twist completion, followed by a deterministic kinematic strategy that executes the reset motion.

1) Switching Discriminator: Transition detection is formulated as binary classification. The discriminator $\mathcal{D}_\phi(o_t) \to \{0, 1\}$ predicts whether twisting has stably completed. We collect twisting trajectories $\tau_{\text{twist}} = \{o_0, \ldots, o_T\}$ from the RL policy, where T denotes the terminal time step of the twist phase. The terminal step is determined by reaching the maximum episode length or observing no rotation increase over a fixed duration.

Let N denote the length of a temporal window at the end of the twist phase (e.g., $N = 5$ steps). Frames within the final N steps, i.e., $t \in [T - N + 1, T]$, are labeled positive

($y_t = 1$), indicating stable twist completion and readiness for reset. All preceding frames $t \in [0, T-N]$ are labeled negative ($y_t = 0$), indicating active twisting. This labeling strategy ensures that the final N-step window captures the stable completion duration and provides a robust trigger signal for the reset phase.

Training follows a two-stage DAgger-inspired pipeline. **Phase 1 (Offline):** The discriminator is initialized with trajectories from the final 20% of RL training iterations, where the twisting policy has converged to stable performance. This behavioral cloning phase provides a strong initial classifier. **Phase 2 (Online):** The discriminator is deployed with the current policy. On-policy samples are collected when predictions diverge from ground truth. Every 50 iterations, these samples are aggregated into the training set, the discriminator is retrained, and the updated model is deployed. This online refinement aligns the discriminator with the evolving policy distribution.

At deployment, reset is triggered when the discriminator predicts $y_t = 1$ for at least $\lceil 0.8N \rceil$ consecutive frames. The relaxed threshold avoids the overly strict requirement that all N frames be positive, where a single misclassification could block switching. This relaxation may trigger reset a few steps before the ground-truth terminal step T, but cap rotation has already stabilized in the late twist phase. The design balances robustness to prediction noise with timely transition to reset.

2) Kinematic Reset Strategy: Once triggered, the reset primitive executes deterministic kinematic motion. Unlike twisting, which requires learned contact dynamics, reset reduces cap contact and recovers a repeatable hand configuration. The motion proceeds in two stages. First, the flexed joints extend to reduce fingertip-cap interference during retraction. Second, the hand interpolates smoothly back to the initial configuration within joint limits. This strategy reduces collision risk and improves state recovery consistency for subsequent cycles.

V. Experiments

This section evaluates DexCapTwist from three aspects. First, DexTwistSet is analyzed to verify asset diversity and simulation feasibility. Second, CycleTwist is compared with monolithic cap-twisting baselines in simulation. Third, the learned policy is deployed on real containers to evaluate zero-shot transfer.

A. DexTwistSet Asset Analysis

1) Dataset Diversity: To construct the asset pool, 5,250 valid bottle contours are collected from web-based sources to form the *Original* subset. ContourVAE then synthesizes an *Interpolation* subset and a *Random* subset, each with 2,625 contour samples. After 3D conversion, grasp initialization, and simulation feasibility validation, DexTwistSet contains 4,015 validated articulated bottle assets.

To evaluate the latent space learned by ContourVAE, the latent variables from all three subsets are projected onto a two-dimensional plane with t-SNE. As shown in Fig. 4, the *Interpolation* subset lies near the distribution covered by the *Original* subset. This confirms its intended role: producing intermediate contours that remain close to empirical container shapes. The projection also shows that interpolation fills sparse regions of the original contour set, while the *Random* subset provides additional samples across the learned latent distribution. Together, the three subsets form a broad and continuous contour pool for asset construction.

The augmented contour pool is converted into USD assets and validated in the LEAP-ROHand simulation environment. Fig. 5 shows examples of the validated bottle assets. The 4,015 validated assets are partitioned into training and testing sets at an 8:2 ratio.

TABLE I
PPO Training Hyperparameters

Parameter	Value
Learning Rate	5×10^{-4}
Discount Factor (γ)	0.99
GAE Parameter (λ)	0.95
Clip Parameter (ϵ)	0.2
Value Loss Coefficient	1.0
Entropy Coefficient	0.0
Number of Learning Epochs	8
Number of Mini-batches	8
Steps per Environment	16
Max Gradient Norm	1.0
Desired KL Divergence	0.008
Max Training Iterations	300
Actor Network Hidden Dims	[256, 128, 64]
Critic Network Hidden Dims	[256, 128, 64]
Activation Function	ELU

B. Simulation Protocol

1) Training Configuration: All policies are trained and evaluated in Isaac Lab. Training uses 4096 parallel environments on a single NVIDIA RTX 3080 GPU for high-throughput contact-rich simulation. Each method is trained with three random seeds (42, 123, 456). The simulation runs at a 60 Hz physics frequency and a 30 Hz control frequency.

Proximal Policy Optimization (PPO) is used as the reinforcement-learning algorithm. Table I lists the hyperparameters. Both actor and critic networks use 3-layer MLPs with hidden dimensions [256, 128, 64] and ELU activations.

2) Robot Configuration: The simulated bimanual system uses two specialized hands mounted on UR3 robot arms. The right hand is a 16-DOF fully actuated LEAP Hand for active twisting. The left hand is an underactuated 6-DOF ROHand (ROH-AP001) for stabilizing the bottle body.

3) Domain Randomization: Domain randomization is applied during training to support zero-shot sim-to-real transfer. Table II lists the randomized parameters, including object dynamics, cap-body joint resistance, hand-object contact friction, and observation noise.

4) Evaluation Protocol: All trained policies are evaluated on the held-out test set using 1024 parallel environments per run. Each episode lasts 200 simulation steps, allowing multiple twist-reset cycles. The test set is grouped into three geometric distributions: *Original*, *Interpolation*, and *Random*. Each method is evaluated on all test bottles in each group,

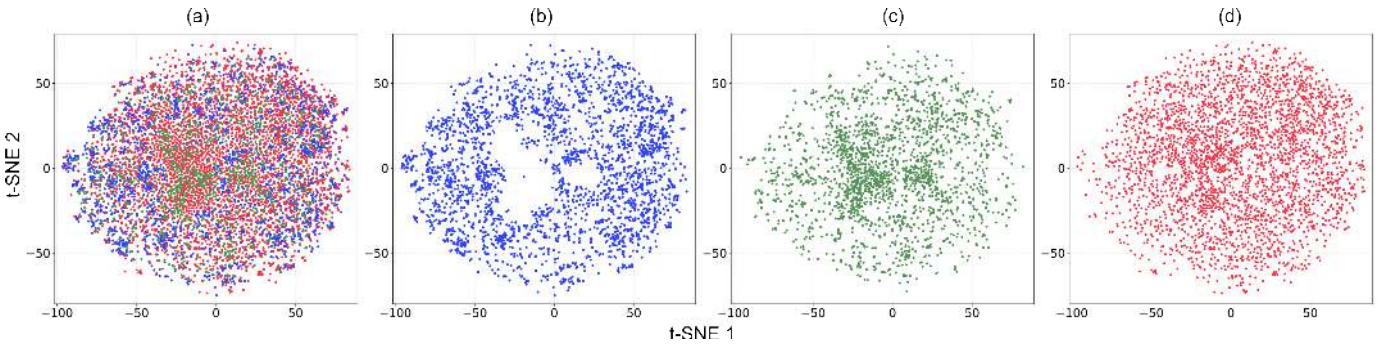

Fig. 4. t-SNE visualization of bottle contour latent distributions. (A) Combined distribution of all three subsets. (B) Original subset. (C) Interpolation subset. (D) Random subset.

Fig. 5. Synthetic bottle assets included in DexTwistSet. The gallery shows geometric and appearance diversity, including varied shapes and material properties instantiated in Isaac Sim.

with results aggregated across the three random seeds. Fig. 6 shows the simulated evaluation set and representative execution snippets.

5) Evaluation Metrics: To quantify manipulation performance, we define three metrics:

- **Total Rotation Angle (TRA)**: The cumulative cap rotation angle (in degrees) achieved within the 200-step evaluation horizon.
- **Effective Rotation Rate (ERR)**: The percentage of episodes where the cap rotation exceeds 120 degrees.
- **Twisting Velocity** (vel): The average rotation angle per environment step, computed as $vel = \mathrm{TRA}/T$, where T is the 200-step evaluation horizon. Higher vel indicates faster twisting execution.

C. Simulation Comparative Evaluation

1) Baselines: We compare CycleTwist with three baselines. **1) Monolithic PPO**: This baseline is derived from CycleTwist by removing the switching discriminator and reset primitive. It trains a single end-to-end policy without an explicit twist-reset phase distinction, testing whether primitive decoupling is necessary for long-horizon manipulation. **2) Twisting Lids Off (TLO) [9]**: TLO is a competitive end-to-end approach

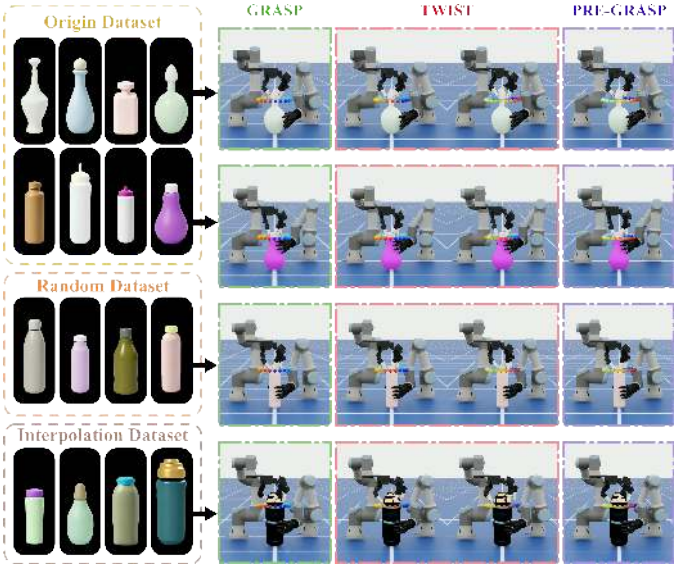

Fig. 6. Simulated evaluation objects and representative trajectories. The left panel shows selected bottle assets from the Original, Interpolation, and Random subsets. The right panel shows representative execution snippets organized by manipulation stage: GRASP, TWIST, and PRE-GRASP.

TABLE II
DOMAIN RANDOMIZATION

Parameter	Range/Value
Object Properties	
Bottle Mass (kg)	[0.03, 0.3]
Cap-Body Joint Friction Scale	[0.3, 2.0]
Hand Properties	
Hand Friction Coefficient	[0.5, 1.5]
Observation Noise	
Bottle Position Noise	$\mathcal{U}(-0.01, 0.01)$
Cap Contour Noise	$\mathcal{U}(-0.01, 0.01)$
Joint Position Noise	$\mathcal{U}(-0.02, 0.02)$
Action Noise	
Action Noise	$\mathcal{N}(0, 0.1)$

for bottle-cap twisting with specialized reward shaping and domain randomization. **3) DexAnyTwist [24]**: DexAnyTwist treats general twisting as hybrid manipulation system identification. Its DexSifter procedure iteratively partitions objects according to expert performance, refines specialized policies on dynamically consistent subsets, and uses a learned router for hard expert selection. We implement the method with the same LEAP action space, PPO budget, domain randomization, and evaluation protocol used by the other baselines. Unlike CycleTwist, DexAnyTwist identifies object-dependent manipulation regimes rather than explicitly decomposing each episode into twisting and reset phases.

2) Results and Analysis: Table III summarizes performance on the held-out DexTwistSet test set. CycleTwist achieves the highest average TRA and twisting velocity across all three geometry groups, while maintaining strong effective-rotation performance. DexAnyTwist improves TRA over Monolithic PPO in all three groups, indicating that expert specialization reduces interference across heterogeneous bottle dynamics; however, its object-level routing does not explicitly address the within-episode twist–reset cycle. On the Original, Interpolation, and Random groups, CycleTwist achieves TRA values of $202.7°$, $210.1°$, and $194.1°$, respectively. Monolithic PPO reaches $163.5°$, $129.0°$, and $159.3°$. DexAnyTwist reaches $172.8°$, $158.6°$, and $166.9°$, respectively, while TLO [9] reaches $103.2°$, $101.5°$, and $114.7°$.

The performance gap indicates that monolithic policies have lower rotation efficiency in cyclic twist-reset manipulation. A single policy must handle contact-rich torque generation and hand recovery at the same time, which makes error accumulation more likely under diverse geometries. CycleTwist improves rotation by using explicit primitive decoupling and learned switching logic. The switching discriminator triggers phase transitions from real-time execution states, helping maintain consistent manipulation quality over repeated cycles.

The illustrative signed rotation trajectories in Fig. 7 further show the behavior implied by the endpoint statistics in Table III. CycleTwist accumulates rotation more rapidly and reaches higher final rotation across all asset categories. Monolithic PPO grows more slowly, while TLO achieves lower final rotation under the same horizon. DexAnyTwist lies between CycleTwist and the monolithic baselines. Temporary reductions in slope represent the population-level effect of contact adjustment, slippage, or hand reset. For CycleTwist and its two ablations, each cycle contains approximately 45–50 twisting steps followed by about 20 reset steps. The twisting slope is initially high and gradually decreases as the hand approaches its contact limit. Rotation remains nearly constant through most of reset, then its slope increases near the end because some bottles finish reset and resume twisting earlier than others. Individual bottles may briefly rotate backward, but averaging across many bottles makes a decrease in the population mean unlikely.

D. Ablation Studies

We conduct ablation studies to evaluate two key design choices in CycleTwist: *w/o Geometry*, which removes bottle geometry inputs including body height and cap contour points, and *w/o Force Reward*, which removes the tangential force term $r_{\text{tangential}}$ from the reward. Results are shown in Table III and the bottom row of Fig. 7.

The *w/o Geometry* variant shows lower TRA, ERR, and twisting velocity, confirming the importance of geometric observations. Without explicit geometry, the policy has reduced ability to adapt grasp configurations to different bottle shapes. The *w/o Force Reward* variant achieves moderate TRA but shows larger variation in the cumulative rotation curves and reduced twisting velocity. This degradation is consistent with more frequent fingertip slippage when tangential contact-force guidance is removed.

The cumulative rotation curves show step-like increases during evaluation. These segments correspond to successful twist-reset cycles. After the switching discriminator triggers reset, the scripted reset returns the hand to a favorable configuration. The next twisting phase then produces another increase in cumulative cap rotation. The complete CycleTwist framework achieves higher final TRA, ERR, and velocity with more consistent cumulative rotation growth, showing that geometry encoding and tangential contact-force shaping are both important for stable dexterous cap manipulation.

E. Switching Discriminator Analysis

1) Evaluation Protocol: We evaluate discriminator performance across three complementary aspects: (1) offline classification accuracy on held-out validation data comprising 500 trajectories per bottle geometry, (2) offline deployment quality via trajectory replay, and (3) online deployment quality during closed-loop execution across 200 test episodes.

Classification metrics assess frame-level binary prediction quality. The final $N = 5$ steps of each trajectory are labeled as "ready for reset" (label 1); all preceding frames are labeled as "still twisting" (label 0).

Deployment quality is evaluated under the temporal consistency rule that requires $\lceil 0.8N \rceil = 4$ consecutive positive predictions before reset. This gating filters transient prediction noise. We measure temporal alignment with the following metrics, where T^* denotes the expected trigger point. It is defined as $T^* = T - 0.2N = T - 1$ to account for the relaxed threshold ($0.8N$ instead of N):

- **Mean Temporal Error (MTE)**: average difference between the actual trigger time and T^*, in simulation steps. Positive values indicate delayed switching.

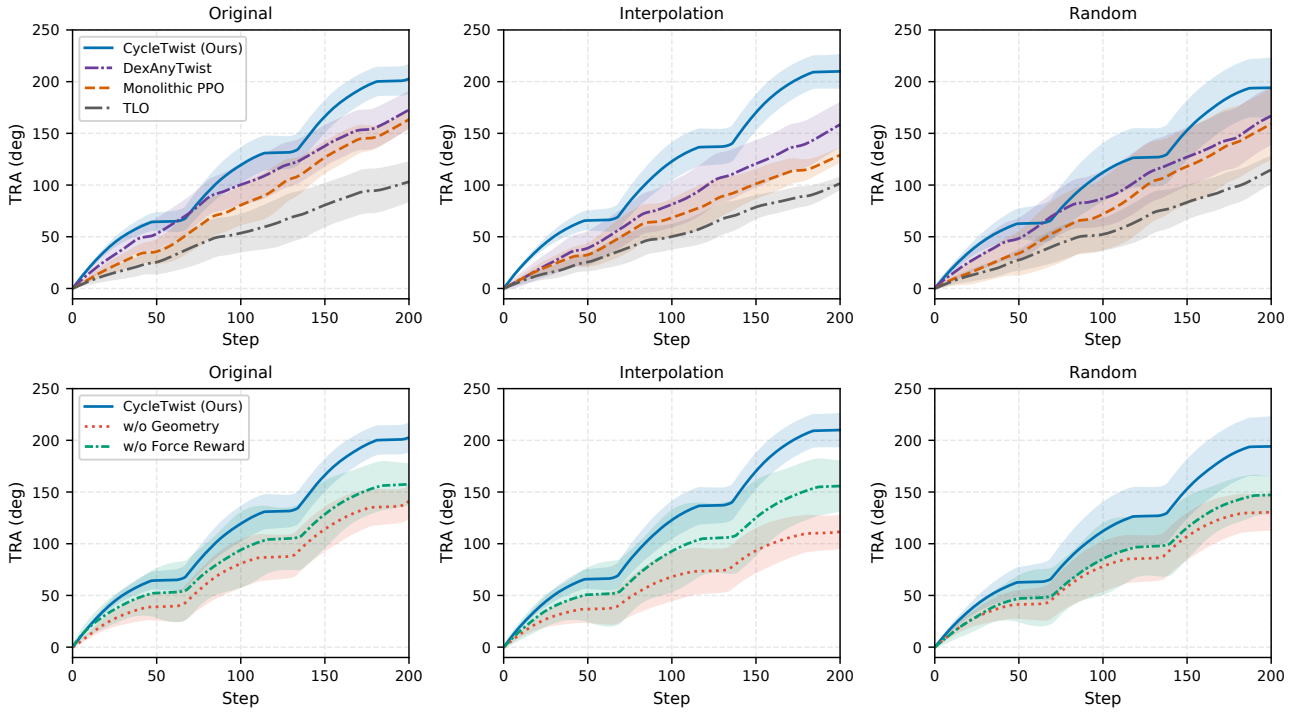

Fig. 7. Illustrative signed cap-rotation trajectories over a 200-step horizon across three asset categories. Top row: baseline comparison against DexAnyTwist, Monolithic PPO, and TLO. Bottom row: ablation study comparing full CycleTwist with w/o Geometry and w/o Force Reward variants. All trajectories are provisional simulations constrained to match the final TRA mean and standard deviation reported in Table III. Contact adjustment and hand reset are represented by temporary reductions in slope rather than decreases in the population mean. CycleTwist and its ablations follow 45–50-step twist and approximately 20-step reset cycles. Each method uses a distinct time-varying uncertainty profile; shaded regions indicate standard deviation across three simulated seeds. These curves will be replaced by measured signed-rotation rollouts.

TABLE III
SIMULATION RESULTS ON DEXTWISTSET.

Method	Original			Interpolation			Random		
	TRA ↑	ERR ↑	*vel* ↑	TRA ↑	ERR ↑	*vel* ↑	TRA ↑	ERR ↑	*vel* ↑
Baseline Comparison									
Monolithic PPO	163.5±8.1	**71.2±17.6**	0.82±0.04	129.0±6.8	50.1±14.6	0.65±0.03	159.3±33.7	69.0±24.8	0.80±0.17
TLO [9]	103.2±18.9	34.8±18.5	0.52±0.09	101.5±5.7	26.3±5.6	0.51±0.03	114.7±12.8	37.8±22.2	0.57±0.06
DexAnyTwist [24]	172.8±17.4	68.5±15.6	0.86±0.09	158.6±20.8	57.4±14.8	0.79±0.10	166.9±27.2	67.2±18.5	0.83±0.14
Ablation Study									
w/o Geometry	141.4±15.4	31.3±3.7	0.71±0.08	111.6±15.7	22.1±5.3	0.56±0.08	130.5±16.9	31.1±16.3	0.65±0.08
w/o Force Reward	157.6±19.6	48.6±14.7	0.79±0.10	155.8±24.1	51.8±15.6	0.78±0.12	147.2±17.0	49.7±17.2	0.74±0.09
CycleTwist (Full)	**202.7±13.6**	69.2±12.6	**1.01±0.07**	**210.1±15.8**	**75.0±13.4**	**1.05±0.08**	**194.1±28.4**	**81.2±20.6**	**0.97±0.14**

- **Premature Switch Rate (PSR)**: percentage of episodes triggering more than one step before T^*.
- **Delayed Switch Rate (DSR)**: percentage of episodes triggering more than one step after T^*, including timeout-triggered episodes.
- **Success Switch Rate (SSR)**: percentage of episodes triggering within ± 1 step of T^*, i.e., $t_{\text{trigger}} \in [T-2, T]$.

2) Results and Analysis: Table IV summarizes the discriminator results for Phase 1, which uses behavioral cloning on final-stage RL trajectories, and Phase 2, which uses DAgger-based refinement with on-policy data aggregation.

Classification Performance. Both training phases achieve strong offline classification performance. Phase 2 improves F1 score from 91.3% to 95.7%. Precision increases from 88.2% to 93.6%, while recall improves from 94.7% to 97.8%. These gains indicate that on-policy data aggregation refines the discriminator's decision boundary.

Offline Deployment Quality. Under temporal consistency gating on validation trajectories, Phase 1 achieves an SSR of 91.2% with an MTE of 0.42 steps. Phase 2 increases SSR to 95.5% and reduces MTE to 0.18 steps, a 57% reduction in temporal error. DSR decreases from 7.2% to 2.6%, indicating more accurate reset timing relative to T^*.

Online Deployment Quality. Under closed-loop execution, DAgger refinement narrows the gap between offline validation and online deployment. Phase 1 exhibits an MTE of 0.94 steps and an SSR of 84.9%. Phase 2 improves these values to 0.33 steps and 91.2%. The DSR reduction from 12.3% to 5.1% demonstrates improved robustness to distribution shift between offline replay and online policy execution.

F. Real-World Experiments

1) Hardware Platform and Deployment Protocol: To validate practical applicability, we construct a real-world dual-

TABLE IV
SWITCHING DISCRIMINATOR EVALUATION.

Metric	Phase 1 (BC)	Phase 2 (DAgger)
Offline Classification		
Accuracy (%)	95.6	97.3
Precision (%)	88.2	93.6
Recall (%)	94.7	97.8
F1 Score (%)	91.3	95.7
Offline Deployment Quality		
Mean Temporal Error (MTE, steps)	0.42	0.18
Premature Switch Rate (PSR, %)	1.6	1.9
Delayed Switch Rate (DSR, %)	7.2	2.6
Success Switch Rate (SSR, %)	91.2	95.5
Online Deployment Quality		
Mean Temporal Error (MTE, steps)	0.94	0.33
Premature Switch Rate (PSR, %)	2.8	3.7
Delayed Switch Rate (DSR, %)	12.3	5.1
Success Switch Rate (SSR, %)	84.9	91.2

arm platform that preserves the functional division between twisting and stabilization. The hardware comprises two UR3 robot arms. A 16-DOF LEAP Hand performs dexterous twisting, while a 6-DOF RH56F1 Inspire Hand stabilizes the bottle body. A RealSense L515 RGB-D camera captures the scene. Grounded-SAM [26] segments the bottle body and cap from images, after which depth is used to recover contours, shapes, scales, and 3D positions. The control policies run at 30 Hz and communicate with the robotic hands through ROS interfaces.

The policy is evaluated on 20 everyday containers spanning five categories: condiment containers, insulated tumblers, dry-storage containers, beverage bottles, and household-care containers. This set covers diverse cap shapes, cap diameters, thread lengths, body compliance, and body geometries. The policy is trained only in Isaac Sim with domain randomization and DexTwistSet geometry variation, then deployed directly on hardware without real-world fine-tuning. A trial is successful if the cap is fully detached from the bottle body without detrimental collisions.

2) Results and Analysis: Fig. 8 summarizes the real-world object set, perception outputs, and representative physical executions. Table V reports per-object quantitative results. CycleTwist achieves a 78.5% average real-world success rate across 200 trials. Category-level performance varies with object geometry. Dry-storage and condiment containers achieve higher success rates because their caps are generally wider and flatter, providing larger contact regions for stable twisting. Beverage bottles have smaller cap radii and require more precise fingertip placement. Insulated tumblers and household-care containers often require longer rotation sequences. These results indicate that CycleTwist maintains effective behavior across diverse cap sizes, thread lengths, and container geometries.

These real-world evaluations demonstrate that the proposed sim-to-real training pipeline maintains effective cap manipulation across diverse categories, geometries, materials, and thread configurations. The results support the practical value of combining DexTwistSet's asset diversity with CycleTwist's hybrid primitive control.

TABLE V
REAL-WORLD EVALUATION RESULTS.

Category	Object	Count	SR	Category Avg.
Condiment Containers	Condiment A	10/10	100%	82.5%
	Condiment B	9/10	90%	
	Condiment C	10/10	100%	
	Condiment D	4/10	40%	
Insulated Tumblers	Tumbler A	10/10	100%	72.5%
	Tumbler B	4/10	40%	
	Tumbler C	9/10	90%	
	Tumbler D	6/10	60%	
Dry-Storage Containers	Dry-Storage A	10/10	100%	90.0%
	Dry-Storage B	8/10	80%	
	Dry-Storage C	10/10	100%	
	Dry-Storage D	8/10	80%	
Beverage Bottles	Beverage A	9/10	90%	67.5%
	Beverage B	5/10	50%	
	Beverage C	8/10	80%	
	Beverage D	5/10	50%	
Household-Care Containers	Household A	9/10	90%	80.0%
	Household B	6/10	60%	
	Household C	9/10	90%	
	Stain Remover	8/10	80%	
Overall Average		157/200	78.5%	78.5%

VI. CONCLUSION

This paper presented **DexCapTwist**, a unified framework for contact-rich, long-horizon bimanual bottle-cap twisting. DexTwistSet is a dataset of diverse, simulation-ready articulated bottle assets constructed from internet images through ContourVAE-based geometry synthesis and BODex-based bimanual grasp initialization. CycleTwist addresses cyclic unscrewing by decomposing control into a learned twisting primitive and a deterministic reset primitive, coordinated by a DAgger-refined Switching Discriminator. This design separates contact-rich torque generation from hand re-positioning, reducing error accumulation during repeated twist-reset cycles.

Simulation experiments across original, interpolated, and randomly sampled bottle geometries show that CycleTwist improves average rotation and twisting velocity over monolithic RL baselines. Ablation and discriminator analyses further verify the importance of geometry-aware observations, tangential contact-force shaping, and accurate phase switching. Real-world experiments on 20 everyday containers demonstrate zero-shot transfer from Isaac Sim to a physical dual-arm platform, supporting the practical value of scalable synthetic asset diversity combined with hybrid primitive control.

Future work will extend the asset pipeline with richer 3D generative models for texture, material, and geometric variation. Tactile sensing, adaptive online parameter estimation, and closed-loop task-progress estimation can also be integrated into CycleTwist for deployment across more hands, object materials, and operating conditions.

REFERENCES

[1] R. Wang et al., *Dexgraspnet: A large-scale robotic dexterous grasp dataset for general objects based on*

Fig. 8. Real-world zero-shot evaluation objects, perception outputs, and physical executions. The left panel presents 20 everyday containers grouped into five categories, with each object shown by its RGB image, segmentation result, and extracted contour. The right panel shows representative execution snippets selected from the five categories, ordered by GRASP, TWIST, and PRE-GRASP stages.

simulation, 2023. arXiv: 2210.02697 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2210.02697.

[2] J. Zhang et al., "Dexgraspnet 2.0: Learning generative dexterous grasping in large-scale synthetic cluttered scenes," in *8th Annual Conference on Robot Learning*, 2024.

[3] J. Ye et al., *Dex1b: Learning with 1b demonstrations for dexterous manipulation*, 2025. arXiv: 2506.17198 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2506.17198.

[4] J. He et al., *Dexvlg: Dexterous vision-language-grasp model at scale*, 2025. arXiv: 2507.02747 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2507.02747.

[5] K. Mo et al., "Partnet: A large-scale benchmark for fine-grained and hierarchical part-level 3d object understanding," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 909–918.

[6] A. Joshi et al., *Infinigen-sim: Procedural generation of articulated simulation assets*, 2025. arXiv: 2505.10755 [cs.GR]. [Online]. Available: https://arxiv.org/abs/2505.10755.

[7] C. Bao, H. Xu, Y. Qin, and X. Wang, "Dexart: Benchmarking generalizable dexterous manipulation with articulated objects," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 21 190–21 200.

[8] Z. Mandi, Y. Hou, D. Fox, Y. Narang, A. Mandlekar, and S. Song, *Dexmachina: Functional retargeting for bimanual dexterous manipulation*, 2025. arXiv: 2505.24853 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2505.24853.

[9] T. Lin, Z.-H. Yin, H. Qi, P. Abbeel, and J. Malik, "Twisting lids off with two hands," in *Conference on Robot Learning*, PMLR, 2024, pp. 5220–5235.

[10] Y. Kim, C. H. Rask, and C. Sloth, *Tac2motion: Contact-aware reinforcement learning with tactile feedback for robotic hand manipulation*, 2025. arXiv: 2509.17812 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2509.17812.

[11] Y.-W. Chao et al., "Dexycb: A benchmark for capturing hand grasping of objects," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 9044–9053.

[12] S. Brahmbhatt, C. Ham, C. C. Kemp, and J. Hays, "Contactdb: Analyzing and predicting grasp contact via thermal imaging," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 8709–8719.

[13] S. Brahmbhatt, C. Tang, C. D. Twigg, C. C. Kemp, and J. Hays, "Contactpose: A dataset of grasps with object contact and hand pose," in *European Conference on Computer Vision*, Springer, 2020, pp. 361–378.

[14] Y. Song et al., "Learning 6-DoF fine-grained grasp detection based on part affordance grounding," *IEEE*

Transactions on Automation Science and Engineering, vol. 22, pp. 15 200–15 214, 2025.

[15] T. Yu et al., "Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning," in *Conference on robot learning*, PMLR, 2020, pp. 1094–1100.

[16] T. Mu et al., "Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

[17] J. Gu et al., "Maniskill2: A unified benchmark for generalizable manipulation skills," in *The Eleventh International Conference on Learning Representations*, 2023.

[18] M. V. Liarokapis and A. M. Dollar, "Combining analytical modeling and learning to simplify dexterous manipulation with adaptive robot hands," *IEEE Transactions on Automation Science and Engineering*, vol. 16, no. 3, pp. 1361–1372, 2019.

[19] X. Sun, J. Li, A. V. Kovalenko, W. Feng, and Y. Ou, "Integrating reinforcement learning and learning from demonstrations to learn nonprehensile manipulation," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 3, pp. 1735–1744, 2023.

[20] M. Ohka, N. Morisawa, and H. B. Yussof, "Trajectory generation of robotic fingers based on tri-axial tactile data for cap screwing task," in *2009 IEEE International Conference on Robotics and Automation*, IEEE, 2009, pp. 883–888.

[21] Y. Cui, T. Matsubara, and K. Sugimoto, "Kernel dynamic policy programming: Applicable reinforcement learning to robot systems with high dimensional states," *Neural Networks*, vol. 94, pp. 13–23, 2017.

[22] F. Li, Y. Bai, M. Zhao, T. Fu, Y. Men, and R. Song, "Research on robot screwing skill method based on demonstration learning," *Sensors*, vol. 24, no. 1, p. 21, 2024.

[23] Y. Lin et al., "Neuraltouch: Neural descriptors for precise sim-to-real tactile robot control," *IEEE/ASME Transactions on Mechatronics*, 2026.

[24] X. Liu et al., "Dexanytwist: Learning general dexterous twisting with hybrid manipulation system identification," *National Science Review*, nwag351, 2026.

[25] D. Lee, C. Li, and D. Lee, *Dextwist: Dexterous hand retargeting for twist motion via mixed reality-based teleoperation*, 2026. arXiv: 2605.12182 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2605.12182.

[26] T. Ren et al., *Grounded sam: Assembling open-world models for diverse visual tasks*, 2024. arXiv: 2401.14159 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2401.14159.

[27] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Computer vision, graphics, and image processing*, vol. 30, no. 1, pp. 32–46, 1985.

[28] Y. Chen et al., *Bodex: Body-object dexterous manipulation*, 2025. arXiv: 2503.16980 [cs.RO].